

SCOS Real-Time

Transition Plan: from Offline Processing to Real-Time Processing
Speckle Contrast Optical Spectroscopy

Project Parameters:

Frame size: 700 × 700 pixels

Frame rate: ~20 Hz

Recording duration: up to several hours

Camera: Basler GigE (via pypylon)

Table of Contents

- Why rewrite the code from scratch
- The main problem of real-time processing (GUI freeze)
- Solution architecture: 3 threads + 2 queues
- Considerations for long recordings (several hours)
- Work plan by phases (5 phases)
- Phase 0 — Preparing the reference
- Phase 1 — Architecture design on paper
- Phase 2 — Fixing tools and dependencies
- Phase 3 — Writing code in layers (5 layers)
- Phase 4 — Extension and polishing
- Checklist: what to do today
- Glossary: what all these words mean
- Clarifications after review: Pylon capture strategy — OneByOne; Overload behavior

1. Why Rewrite the Code from Scratch

What We Are Doing

We are writing a new system according to the docs/SCOS_protocol.md protocol, in which:

- Frame capture from the camera runs continuously in a separate thread
- Processing runs in parallel in another thread, without blocking capture
- The GUI stays responsive at all times, updated by a timer
- All protocol stages (4 calibration and measurement stages) are implemented as a state machine
- Results are periodically saved to disk to prevent data loss

What We Are NOT Discarding

The existing infrastructure remains:

- camera.py — wrapper over pypylon. Will be used as-is, possibly with minimal changes
- Math in processor.py — κ^2 formulas with noise correction. We will copy them into a new clean module
- convert_gain — function for converting dB to DU/e. Possibly extended to load from a table (as requested in the protocol)
- Tests in tests/ — we will extend, not discard
- scos_config.json — we will add new parameters (window_size, N1, N2, recording_length, etc.)

2. The Main Problem of Real-Time Processing

The main problem of a real-time GUI application is GUI freeze. This happens when the main thread of the program does something time-consuming and cannot respond to the user during that time.

Three Tasks That Must Run Simultaneously

In our application, three different jobs must execute at the same time:

Task A — Frame Capture

- The camera delivers a frame every ~50 ms (at 20 Hz)
- Every frame must be retrieved in time, otherwise it will be lost
- Capture must run continuously, without pauses

Task B — Frame Processing

- For each frame, κ^2 must be calculated with noise correction
- This is an expensive operation — takes milliseconds (we will measure exactly later)
- If processing cannot keep up — under no circumstances discard frames. Detailed behavior is described in section 13.2

Task C — Display

- Redraw the graph ($1/\kappa^2$ vs. time)
- Show the current frame as an image (every 2.5 s per protocol)
- Respond to buttons, sliders, checkboxes
- Must be 'responsive' — a delay > 1000 ms is already perceptible to the user

3. Solution Architecture: 3 Threads + 2 Queues

The standard solution for a real-time GUI is to split work into three threads that communicate through queues. This pattern is known as producer-consumer.

Architecture Diagram

```
Thread 1 (Camera) → [frame queue, size=20] → Thread 2 (Processor) → [result queue, size=∞] → GUI (QTimer every 100ms)
```

Thread 1: Frame Capture

This is a special object — QThread (or regular threading.Thread). Its job:

- Open the camera via pypylon
- In a loop, retrieve frames with camera.RetrieveResult()
- Put each frame into frame_queue
- If the queue is full — do NOT discard the frame. Pause capture, raise a warning, and ask the user (see section 13.2). This rule is the same for all states.

Queue size = 20 for all states (preview, calibration, measurement).

If the queue is small (2), the 80%-full alarm triggers at 1.6 frames in the queue — meaning 80 ms before overflow at 20 Hz. That is critically little: the user physically cannot read the dialog and press a button in time. With size 20, the alarm at 80% = 16 frames — about 800 ms for the user to react.

Memory-wise this is nothing: 20 frames × 1 MB ≈ 20 MB. Details — in section 13.2.

IMPORTANT to prevent dropping frames:

Use Pylon GrabStrategy_OneByOne with MaxNumBuffer = 20 and track GetNumberOfSkippedImages(). Detailed rationale — in section 13.1.

Thread 2: Frame Processing

Also a separate thread. Its job:

- Take a frame from frame_queue (blocking get with timeout)
- Calculate κ^2 with noise correction according to protocol formulas
- Put the result into result_queue
- React to state changes (DARK_CAL → BRIGHT_CAL → MEASURING)

What does a result contain?

```
@dataclass
class ProcessingResult:
```

```

timestamp: float          # time from start of measurement, seconds
kappa2_raw: float         # raw  $\kappa^2$  (without corrections)
    kappa2_fixed: float    # with noise correction
        bfi: float        # 1 / kappa2_fixed
rbfi: float | None        # normalized (if mean_BFI_first is already known)
    mean_intensity: float  # <I> in ROI
frame_number: int         # sequential frame number

```

Thread 3 (Main): GUI

In Qt this is always the main thread. Its rules:

- Never block — no while True loops, no sleep, no heavy computations
- Use QTimer.timeout signal, which calls a function every N ms
- This function reads result_queue (non-blocking get), takes everything available, updates the graph
- Responds to signals from other threads (via pyqtSignal — this is a thread-safe mechanism)

How to update the graph without lag: built-in downsampling in pyqtgraph

pyqtgraph already has built-in automatic point downsampling. One line of code is enough — no need to build your own 'two levels of data' system manually:

```

self.graph.setDownsampling(auto=True, mode='peak')
self.graph.setClipToView(True)

```

Queues and Locking

What is queue.Queue in Python

This is a special thread-safe container. Its methods:

- queue.put(item) — puts an item. If the queue is full, waits for space
- queue.put(item, block=False) — puts or immediately raises Full error
- queue.get() — retrieves an item. If empty, waits
- queue.get(timeout=1.0) — waits at most 1 second, then raises Empty error

Alternative: collections.deque

If you need 'always fresh' behavior (automatically discard old items when full), collections.deque(maxlen=2) is more convenient. But deque does not block — you have to do polling.

For our pipeline this is fine.

4. Considerations for Long Recordings (Several Hours)

A multi-hour recording changes many design decisions. Simple math:

Data Volumes

Parameter	Value
Recording duration	up to 4 hours
Frame rate	20 Hz
Total frames	up to 288,000
Single frame size	700 × 700 × 2 bytes ≈ 1 MB (Mono12 in 16 bits)
If saving all frames	up to 280 GB
κ^2 graph points	up to 288,000
Memory for κ^2 , BFI, rBFI	288,000 × 3 × 8 bytes ≈ 7 MB — negligible

Problem 1: Saving Frames

The protocol has a 'Save Frames' option. But 280 GB per recording is excessive.

Fastest option:

- RAW / BIN / Dump (unstructured binary stream)
- Just write frame bytes as-is — no compression, no headers, no conversion
- Usually the fastest possible write method
- One big file is faster than thousands of PNG/TIFF/JPEG — fewer open/close operations
- Ring buffer + async writer: camera writes to RAM, a separate thread flushes to SSD. Often more important than the choice of format

Ideal scheme for speckle:

16-bit RAW frames → NVMe SSD → then offline convert to HDF5/TIFF

Additional Solutions:

- Do not save by default (checkbox unchecked)
- If enabled — save only N1 (dark) and N2 (bright) calibration frames. They are needed for subsequent analysis. That is 600+600 = 1200 frames × 1 MB = ~1.2 GB — acceptable
- For the main recording — save only every K-th frame (subsample). Parameter in settings
- Disk space limit: check free space, stop if less than X GB
- Use a compressed format if saving a lot is important (TIFF LZW or HDF5 chunked + gzip)

Problem 2: Graph with 100k+ Points

pyqtgraph is accelerated via OpenGL, but 200,000 points still lag when redrawing.

Solutions:

pyqtgraph has built-in automatic downsampling. One line of code does the job:

```
self.graph.setDownsampling(auto=True, mode='peak')
self.graph.setClipToView(True)
```

Problem 3: Normalization to 'First N Seconds'

Per protocol: $rBFI = BFI / \text{mean_BFI_firstSeconds}$. But over 4 hours the laser may drift, the detector may heat up, the subject may shift. Hard anchoring to the start may give a misleading picture.

Options to discuss with supervisor:

- Keep as in protocol — normalization only on first N seconds
- Rolling normalization: every M minutes re-normalize on the 'reference first N seconds' that are stably low
- Save raw BFI and do not normalize in real-time, but do it offline

This is a question for the scientific supervisor — I should not make this decision. Just note that for long recordings this needs to be discussed.

Problem 4: Periodic Saving

If the program crashes after 3.5 hours of measurement, all data will be lost. Solution:

- Use ONLY HDF5 (.h5) format — it supports append (adding new data without rewriting the whole file). The old .mat v5 format written by `scipy.io.savemat` does NOT support this — so it does not suit us.
- Every M minutes (e.g., 5) APPEND new points (t, BFI, rBFI, mean_intensity) to ONE shared session .h5 file — not create a bunch of small files.
- On program close — final save of the remainder (via `closeEvent` in `QMainWindow`).

Problem 5: Connection Stability

Over several hours anything can happen: a USB cable gets bumped, the network blinks (GigE cameras over network), Arduino reboots. The program should:

- Log everything that happened (logging module)
- On loss of camera connection — try to reconnect, do not crash
- On detecting a problem — save already accumulated data
- Have a 'system health' indicator in the GUI: camera status, dropped frames count, time statistics

5. Work Plan by Phases

The project is divided into 5 phases, each with its own goal. Below is a general overview; details are in the following sections.

Phase	Goal
0. Preparing the reference	Get the 'correct' κ^2 output on a real recording via offline code. This is the reference for verifying real-time code.
1. Architecture on paper	Answer design questions BEFORE coding. Document docs/realtime_architecture.md
2. Fixing tools	pyproject.toml + lock file + ruff/mypy + pre-commit. So that new code is clean from the start.
3. Writing code in layers	Bottom-up: math $\rightarrow$ frame source $\rightarrow$ session $\rightarrow$ threads $\rightarrow$ GUI. Each layer tested independently.
4. Polish	CI, extended tests, README, logging, Claude Code hooks.

Principles Throughout the Plan

- Small Git commits: one change = one commit
- Every commit must leave working code (can build, can run tests)
- Test first, then code (when possible). This is test-driven development in simple form
- Do not optimize before measuring. First write clearly, then profile, then optimize
- Log everything: logging module, not print()

6. Phase 0 — Preparing the Reference

Duration: 1 day. Goal: have concrete κ^2 numbers for a specific recording, against which the new code will be verified.

Step 0.1 — Choose One Real Recording

Find in the lab archive one good recording with the following properties:

- Laser was stably on the entire time
- Object (subject's head / phantom) did not move abruptly
- ROI is well-defined and does not change
- Duration 1–5 minutes (not hours — enough for testing)
- No anomalies like flashes, artifacts, noise from neighboring equipment
- Capture parameters are known: gain, exposure, frame rate, sat capacity

This recording becomes the 'reference' for the coming weeks. Do not lose it, keep a copy in a separate folder.

Step 0.2 — Run the Offline Code

Run the existing offline code (PlotSCOSvsTime.m in MATLAB or Python processor.py in folder-processing mode) on this recording. Save all output:

- Time vector t (in seconds from start)
- κ^2 vector (with noise correction) for each frame
- BFI = $1/\kappa^2$ vector
- rBFI vector (if normalization is performed)

- Processing parameters (window_size, gain, sat_capacity, etc.)

Save as offline_reference.npz:

```
# scripts/save_offline_reference.py
import numpy as np
from processor import SCOSProcessor

# Load frames from folder
frames = load_tiff_folder("path/to/recording") # shape (N, H, W)

# ROI mask (same as during original recording)
mask = make_roi_mask(...)

# Run processing
proc = SCOSProcessor(window_size=7, gain_db=8.0, bit_depth=12)
times, k2_raw, k2_fixed, mean_i = [], [], [], []

frame_rate = 20.0
for i, f in enumerate(frames):
    raw, fixed, m = proc.process(f, mask)
    times.append(i / frame_rate)
    k2_raw.append(raw)
    k2_fixed.append(fixed)
    mean_i.append(m)

bfi = 1.0 / np.array(k2_fixed)

np.savez("tests/data/offline_reference.npz",
        times=np.array(times),
        kappa2_raw=np.array(k2_raw),
        kappa2_fixed=np.array(k2_fixed),
        bfi=bfi,
        mean_intensity=np.array(mean_i),
        window_size=7, gain_db=8.0, bit_depth=12,
        frame_rate=frame_rate)
print(f"Saved {len(times)} reference points")
```

Step 0.3 — Save Part of Frames for Tests

Save the first 20 frames of the same recording into a separate .npz for unit tests:

```
# Same frames as in the reference — for small fast tests
sample = frames[:20] # shape (20, 700, 700)
np.savez("tests/data/sample_frames.npz",
        frames=sample, mask=mask)
```

Step 0.4 — Protocol Gap Map

Read docs/SCOS_protocol.md again and compile a table of what the offline code already does and what it does not:

Protocol Stage	What We Have
κ^2 with shot+dark+quant noise correction	Present in processor.py
Getting G[DU/e] from table	NO — only convert_gain() with hardcoded 10500
Dark frame calibration (N1=600)	NO — only calibration from array in processor.calibrate()
Bright frame calibration (N2=600)	NO
var_dark spatial filter	PARTIAL — no filtering with window size
var_bright spatial filter	NO

Reduce ROI by window/2 + 1	NO
Normalize rBFI to first N seconds	NO
Progressive disk saving	NO
State machine (DARK→BRIGHT→MEAS)	NO
Pop-up 'turn off laser'	NO

This table is your work map. Each 'NO' is a new module or function to write.

7. Phase 1 — Architecture Design on Paper

Questions About Data

- What frame format? — 700×700 Mono12 in uint16, 1 MB
- How many frames per second? — 20 Hz (can raise to 40 if we keep up)
- How many frames kept in memory? — NONE. Only the result after process()
- How many results in memory? — 200,000 × ~50 bytes = 10 MB — all results in memory
- When and where to flush to disk? — every 5 minutes into .h5 (HDF5 supports append)

These are the data types we will use:

Data Type	Examples from Protocol	Best Format	Why
1. Raw camera frames	Dark N1, Bright N2, optional saved SCOS frames	Binary RAW (.bin) / HDF5	Fastest sustained writing. HDF5 if you need indexing/timestamps.
2. Calibration images	mean_dark, var_dark, var_bright	HDF5 or NumPy (.npy)	Fast load/save for arrays, no MATLAB dependency.
3. Time-series signals	BFI, rBFI, <I>	HDF5	Supports append; handles large arrays well.
4. Metadata / settings	Window size, N1, N2, ROI, G, duration	JSON	Human-readable, editable, portable.
5. Graph output	Final graph	PNG + interactive HTML	PNG for reports, HTML for zooming.
6. Session package	Entire experiment	One folder or one HDF5 file	Keeps everything together.

Questions About Computation

- How many ms does process() take for one 700×700 frame with window=7? — UNKNOWN, need to measure
- Budget per frame — 1000/20 = 50 ms
- Where to run bench_processor.py — on the target PC (the one the program will run on)
- What to do if we cannot fit in: (a) reduce ROI, (b) reduce FPS, (c) every other frame, (d) numba/GPU

Questions About Synchronization

- Which threads to use? — CameraThread (QThread), ProcessorThread (QThread), main (GUI)
- How do they communicate? — frame_queue: queue.Queue(maxsize=20), result_queue: queue.Queue(maxsize=0)
- How does GUI learn about new results? — QTimer every 1000 ms reads result_queue
- How to stop everything cleanly? — shared threading.Event() + close()

Questions About the State Machine

States of a SCOS session:

State	What Happens
IDLE	nothing, waiting for Start command
DARK_CAL	capture N1 frames without laser → mean_dark, var_dark
BRIGHT_CAL	capture N2 frames with laser and without object → var_bright
MEASURING_INIT	first M seconds — collecting BFI for normalization
MEASURING	main loop — computing rBFI and drawing graph
FINISHED	saving result to disk, showing message
ERROR	error, need to restart

Questions About GUI

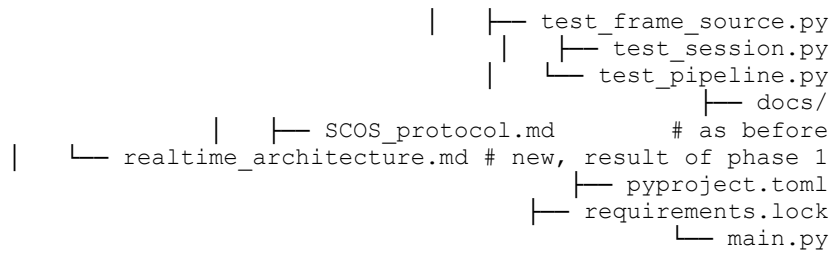
- What widgets are needed? — image, plot, start/stop buttons, progress bar for calibration
- How often is the graph updated? — every 1000 ms by timer
- How often is the image updated? — every 2.5 s (per protocol) during MEASURING; 30 FPS during warmup/IDLE
- What happens when Stop is pressed during DARK_CAL? — dialog 'Abort calibration?', on confirmation — FINISHED state, save what we have

Project Folder Structure

```

scos-python/
├── core/
│   ├── scos_math.py      # k² formulas, shot/dark/quant noise
│   ├── frame_source.py   # abstraction: camera or folder
│   │   ├── camera_source.py # implementation for Basler
│   │   └── folder_source.py # implementation for testing
│   ├── session.py        # state machine
│   ├── pipeline.py       # threads + queues
│   └── recorder.py       # disk saving
├── gui/
│   ├── main_window.py    # main window
│   ├── image_widget.py
│   ├── plot_widget.py
│   └── controls_panel.py # buttons, sliders, checkboxes
├── status_bar.py
├── arduino_uploader.py   # as before
├── tests/
│   └── data/
│       ├── offline_reference.npz
│       ├── sample_frames.npz
│       └── golden_frames.npz # later
└── test_scos_math.py

```



What to Write in docs/realtime_architecture.md

This document is the result of Phase 1. Structure:

- Brief task description (1 paragraph)
- System parameters: frame size, FPS, duration
- Architecture diagram: 3 threads + 2 queues
- Description of each thread: what it does, how it reacts to stop
- State machine: list of states, transitions
- Data structures: ProcessingResult, ConfigParams, SessionStats
- Save protocol: file format, frequency, what to write
- Open questions (things to discuss with supervisor)

8. Phase 2 — Fixing Tools and Dependencies

Duration: half a day. Goal: ensure that new code is written in a clean environment with automatic checks from the very start.

Step 2.1 — Create pyproject.toml

pyproject.toml is the modern standard for Python projects. Replaces setup.py + requirements.txt + a bunch of ad-hoc configs.

```
[project]
name = "scos-python"
version = "0.2.0"
description = "Real-time SCOS for cerebral blood flow"
requires-python = ">=3.11"
dependencies = [
    "pypylon",
    "PyQt6",
    "pyqtgraph",
    "numpy",
    "scipy",
    "opencv-python",
    "pyserial",
    "tiffiffle",
    "h5py",          # for progressive saving
]

[project.optional-dependencies]
dev = [
    "pytest>=8",
    "pytest-cov",
    "pytest-qt",
    "hypothesis",
    "ruff",
    "mypy",
    "pre-commit",
    "pip-tools",
]

[tool.ruff]
line-length = 100
```

```

target-version = "py311"

[tool.ruff.lint]
select = ["E", "F", "I", "B", "UP", "SIM", "PERF", "NPY", "RUF"]
ignore = ["E501"]

[tool.mypy]
python_version = "3.11"
strict = true
files = ["core/"]

[tool.pytest.ini_options]
testpaths = ["tests"]
addopts = "-q"

```

Step 2.2 — Generate requirements.lock

Lock exact versions of all dependencies (including transitive):

```

pip install pip-tools
pip-compile --generate-hashes pyproject.toml -o requirements.lock

```

Commit requirements.lock. Now anyone can reproduce the exact environment:

```

pip install -r requirements.lock

```

This is critical for scientific reproducibility. If scipy releases an update tomorrow, your measurements made today will remain reproducible.

Step 2.3 — Install ruff and mypy

ruff — linter (finds errors and style problems) + formatter. mypy — type checking.

```

pip install -e ".[dev]"
ruff check .
ruff format .
mypy core/

```

Step 2.4 — Install pre-commit

This is a system that runs checks before every git commit. If checks fail — the commit is blocked until you fix them.

File .pre-commit-config.yaml:

```

repos:
- repo: https://github.com/astral-sh/ruff-pre-commit
  rev: v0.7.0
  hooks:
    - id: ruff
      args: [--fix]
    - id: ruff-format

- repo: https://github.com/pre-commit/mirrors-mypy
  rev: v1.13.0
  hooks:
    - id: mypy
      files: ^core/

- repo: local
  hooks:
    - id: pytest-fast
      name: pytest fast
      entry: pytest -q -m "not slow and not camera"
      language: system
      pass_filenames: false
      always_run: true

```

Installation:

```

pip install pre-commit
pre-commit install

```

After this, every git commit will automatically:

- Run ruff check and fix simple errors
- Run ruff format
- Run mypy on the core/ folder
- Run fast tests (without slow and camera marks)

9. Phase 3 — Writing Code in Layers

Duration: main work, 2-4 weeks. Goal: write new code bottom-up, from clean math to GUI.

Principle: Bottom-Up

Do not start with GUI. Start with math, then build everything else on top of it.

- Layer 1: Pure math (no camera, no GUI, no threads)
- Layer 2: Frame source (abstraction: camera or folder)
- Layer 3: Session manager (state machine)
- Layer 4: Threads and queues (real-time pipeline)
- Layer 5: GUI

Each layer can be developed and tested independently of the layers above. This is the main advantage of this approach — you do not write 1000 lines and then run everything and discover nothing works.

Layer 1: core/scos_math.py

What it does: pure functions for calculating κ^2 .

What it does NOT do: knows nothing about camera, threads, GUI, files.

```
# core/scos_math.py
"""Pure SCOS math functions. No state, no I/O."""
import numpy as np
from scipy.ndimage import uniform_filter

def convert_gain_db_to_due(gain_db: float, bit_depth: int,
                           sat_capacity_e: float) -> float:
    """Convert camera gain from dB to DU/e (digital units per electron).

    Parameters
    -----
        gain_db : dB, as shown in Pylon
        bit_depth : 8, 10, or 12 (for Mono8/10/12)
        sat_capacity_e : pixel saturation in electrons (for Basler ace ~10500)

    Returns: DU/e
    """
    G0 = (2 ** bit_depth) / sat_capacity_e
    return G0 * 10 ** (gain_db / 20.0)

def local_mean_and_variance(im: np.ndarray, window: int) -> tuple[np.ndarray,
                                                                    np.ndarray]:
    """Local mean and local variance over a window x window area."""
    im = im.astype(np.float64)
    mean_im = uniform_filter(im, size=window)
    var_im = uniform_filter(im ** 2, size=window) - mean_im ** 2
    var_im = np.maximum(var_im, 0) # numerical protection
    return mean_im, var_im

def shrink_mask_for_window(mask: np.ndarray, window: int) -> np.ndarray:
    """Shrink mask by ceil(window/2)+1 pixels per protocol.

    Why: uniform_filter at ROI borders computes using pixels outside ROI,
    which corrupts  $\kappa^2$ . Shrink the mask so the window lies fully inside.
    """
    from scipy.ndimage import binary_erosion
```

```

        pad = window // 2 + 1
        structure = np.ones((3, 3), dtype=bool)
        return binary_erosion(mask, structure=structure, iterations=pad)

    def compute_kappa2(
        frame: np.ndarray,
        mask: np.ndarray,
        window: int,
        mean_dark: np.ndarray,
        var_dark_filtered: np.ndarray,
        var_bright_filtered: np.ndarray,
        G_due: float,
    ) -> tuple[float, float, float]:
        """One frame → ( $\kappa^2_{\text{raw}}$ ,  $\kappa^2_{\text{corrected}}$ ,  $\langle I \rangle$ ).

        Formula from protocol:
        
$$\kappa^2_{\text{fixed}} = (\text{var\_raw} - \text{var\_dark} - \text{var\_bright} - G \cdot \langle I \rangle - 1/12) / \langle I \rangle^2$$

        """
        im = frame.astype(np.float64) - mean_dark
        mean_im, var_im = local_mean_and_variance(im, window)

        safe = mean_im > 0
        mean_sq = mean_im ** 2

        # Raw  $\kappa^2$ 
        k2_raw_map = np.where(safe, var_im / mean_sq, np.nan)
        k2_raw = float(np.nanmean(k2_raw_map[mask]))

        # With correction
        quant = 1.0 / 12.0
        shot = G_due * mean_im
        corr_num = var_im - var_dark_filtered - var_bright_filtered - shot - quant
        k2_fixed_map = np.where(safe, corr_num / mean_sq, np.nan)
        k2_fixed = float(np.nanmean(k2_fixed_map[mask]))

        mean_intensity = float(np.mean(im[mask]))
        return k2_raw, k2_fixed, mean_intensity

```

Test for Layer 1: tests/test_scos_math.py

```

import numpy as np
from core.scos_math import compute_kappa2, convert_gain_db_to_due

def test_offline_reference_matches():
    """Real-time math must give the same numbers as the offline code.

    We load 20 sample frames, compute  $\kappa^2$  for each with the new code,
    and compare to offline_reference.npz.
    """
    samples = np.load("tests/data/sample_frames.npz")
    ref = np.load("tests/data/offline_reference.npz")

    frames = samples["frames"]
    mask = samples["mask"]
    G = convert_gain_db_to_due(
        gain_db=float(ref["gain_db"]),
        bit_depth=int(ref["bit_depth"]),
        sat_capacity_e=10500.0,
    )

    # No dark/bright calibrations yet, set to zeros
    mean_dark = np.zeros_like(frames[0], dtype=np.float64)
    var_dark = np.zeros_like(frames[0], dtype=np.float64)
    var_bright = np.zeros_like(frames[0], dtype=np.float64)

    for i, f in enumerate(frames[:20]):

```

```

_, k2_fixed, _ = compute_kappa2(
    f, mask, int(ref["window_size"]),
    mean_dark, var_dark, var_bright, G
)
# offline_reference also has no calibrations – must match
assert k2_fixed == pytest.approx(ref["kappa2_fixed"][i], rel=1e-6)

```

Layer 2: core/frame_source.py

Why: a uniform interface for camera and folder. This allows testing EVERYTHING else (threads, session, GUI) without a connected camera.

```

# core/frame_source.py
from abc import ABC, abstractmethod
import numpy as np

class FrameSource(ABC):
    """Base interface for a frame source."""

    @abstractmethod
    def open(self) -> None:
        """Open the source (camera or folder)."""

    @abstractmethod
    def get_frame(self, timeout_s: float = 2.0) -> np.ndarray:
        """Get the next frame. May raise TimeoutError."""

    @abstractmethod
    def close(self) -> None:
        """Close the source."""

    @property
    @abstractmethod
    def frame_rate_hz(self) -> float:
        """Real frame rate (for FolderSource – needs to be simulated)."""

# core/camera_source.py
class CameraFrameSource(FrameSource):
    """Real Basler camera via pypylon."""
    def __init__(self, exposure_us, gain_db, frame_rate_hz, pixel_format):
        # ... analogous to CameraThread.open()
        ...
    def get_frame(self, timeout_s=2.0):
        result = self.camera.RetrieveResult(int(timeout_s*1000),
            pylon.TimeoutHandling_ThrowException)
        if not result.GrabSucceeded():
            raise RuntimeError(result.ErrorDescription)
        img = result.Array.copy()
        result.Release()
        return img

# core/folder_source.py
import time
from pathlib import Path
import tifffile

class FolderFrameSource(FrameSource):
    """Reads .tiff frames from a folder in name order, emulating a real camera.

    Why: you can debug the entire real-time logic (threads, GUI, saving)
    without a camera, just by feeding it a folder with a recording.
    """
    def __init__(self, folder: Path, frame_rate_hz: float = 20.0,
        loop: bool = False):
        self.folder = Path(folder)

```

```

        self._frame_rate = frame_rate_hz
        self._loop = loop
        self._files: list[Path] = []
        self._idx = 0
        self._last_t = None

        def open(self):
            self._files = sorted(self.folder.glob("*.tiff"))
            if not self._files:
                raise FileNotFoundError(f"No .tiff in {self.folder}")
            self._idx = 0
            self._last_t = None

        def get_frame(self, timeout_s=2.0):
            if self._idx >= len(self._files):
                if self._loop:
                    self._idx = 0
                else:
                    raise StopIteration("No more frames in folder")

            # Emulate real time: wait for the interval between frames
            now = time.monotonic()
            if self._last_t is not None:
                elapsed = now - self._last_t
            wait = (1.0 / self._frame_rate) - elapsed
            if wait > 0:
                time.sleep(wait)
            self._last_t = time.monotonic()

            frame = tifffile.imread(self._files[self._idx])
            self._idx += 1
            return frame

        def close(self):
            pass

        @property
        def frame_rate_hz(self):
            return self._frame_rate

    def test_folder_source_yields_frames(tmp_path):
        # Create 5 fake .tiff files
        for i in range(5):
            tifffile.imwrite(tmp_path / f"{i:04d}.tiff", np.full((100,100), i,
                dtype=np.uint16))

    src = FolderFrameSource(tmp_path, frame_rate_hz=100.0) # fast for test
    src.open()
    frames = [src.get_frame() for _ in range(5)]
    src.close()

    assert len(frames) == 5
    for i, f in enumerate(frames):
        assert f.mean() == i # values match

```

Test for Layer 2:

Layer 3: core/session.py

State machine for the SCOS protocol.

```

# core/session.py
from enum import Enum, auto
from dataclasses import dataclass, field
import numpy as np
from core.scos_math import compute_kappa2, local_mean_and_variance

```

```

class State(Enum):
    IDLE = auto()
    DARK_CAL = auto()
    BRIGHT_CAL = auto()
    MEASURING_INIT = auto() # first N sec, collecting for normalization
    MEASURING = auto()
    FINISHED = auto()
    ERROR = auto()

    @dataclass
    class SessionConfig:
        window_size: int = 7
        n_dark_frames: int = 600
        n_bright_frames: int = 600
        recording_minutes: float = 5.0 # inf = indefinite
        norm_seconds: float = 5.0 # how many sec to collect for mean_BFI
        save_frames: bool = False
        output_folder: str = ""
        G_due: float = 0.0 # from table by camera SN + pixel_format

    @dataclass
    class SessionState:
        state: State = State.IDLE
        config: SessionConfig = field(default_factory=SessionConfig)
        mean_dark: np.ndarray | None = None
        var_dark_filtered: np.ndarray | None = None
        var_bright_filtered: np.ndarray | None = None
        dark_buffer: list = field(default_factory=list)
        bright_buffer: list = field(default_factory=list)
        init_bfi_buffer: list = field(default_factory=list)
        mean_bfi_first: float | None = None
        times: list = field(default_factory=list)
        rbfi: list = field(default_factory=list)
        bfi: list = field(default_factory=list)
        mean_intensity: list = field(default_factory=list)
        frames_processed: int = 0
        frames_dropped: int = 0

    class SCOSSession:
        def __init__(self, config: SessionConfig, mask: np.ndarray):
            self.config = config
            self.mask = mask
            self.state = SessionState(config=config)

            def start(self):
                self.state.state = State.DARK_CAL
                # GUI should show pop-up "Turn off laser"

        def feed_frame(self, frame: np.ndarray, t_seconds: float):
            s = self.state
            if s.state == State.DARK_CAL:
                s.dark_buffer.append(frame.astype(np.float64))
                if len(s.dark_buffer) >= self.config.n_dark_frames:
                    self._finish_dark_cal()

            elif s.state == State.BRIGHT_CAL:
                s.bright_buffer.append(frame.astype(np.float64))
                if len(s.bright_buffer) >= self.config.n_bright_frames:
                    self._finish_bright_cal()

            elif s.state in (State.MEASURING_INIT, State.MEASURING):
                self._process_measurement(frame, t_seconds)

            def _finish_dark_cal(self):
                stack = np.stack(self.state.dark_buffer, axis=2)

```

```

        self.state.mean_dark = stack.mean(axis=2)
        var_dark = stack.var(axis=2)
        # Spatial mean filter with window size
        from scipy.ndimage import uniform_filter
        self.state.var_dark_filtered = uniform_filter(var_dark,
                                                    size=self.config.window_size)
        self.state.dark_buffer.clear()
        self.state.state = State.BRIGHT_CAL
        # GUI shows pop-up "Turn on laser, remove object"

        def _finish_bright_cal(self):
            stack = np.stack(self.state.bright_buffer, axis=2)
            # var_bright = ? per protocol - mean spatial filter of the mean
            # ... details depend on what exactly the protocol requires
            ...
            self.state.bright_buffer.clear()
            self.state.state = State.MEASURING_INIT

        def _process_measurement(self, frame, t):
            s = self.state
            _, k2_fixed, mean_i = compute_kappa2(
                frame, self.mask, self.config.window_size,
                s.mean_dark, s.var_dark_filtered, s.var_bright_filtered,
                self.config.G_due)

            if k2_fixed <= 0:
                return
            bfi = 1.0 / k2_fixed

            if s.state == State.MEASURING_INIT:
                s.init_bfi_buffer.append(bfi)
                if t >= self.config.norm_seconds:
                    s.mean_bfi_first = float(np.mean(s.init_bfi_buffer))
                    s.state = State.MEASURING

            if s.mean_bfi_first is not None:
                rbfi = bfi / s.mean_bfi_first
                s.times.append(t)
                s.bfi.append(bfi)
                s.rbfi.append(rbfi)
                s.mean_intensity.append(mean_i)
                s.frames_processed += 1

```

Layer 4: core/pipeline.py

Threads and queues.

```

# core/pipeline.py
import threading
import queue
import time
import numpy as np
from PyQt6.QtCore import QObject, pyqtSignal
from core.frame_source import FrameSource
from core.session import SCOSSession

class RealtimePipeline(QObject):
    """Manages two threads: capture + processing.

    GUI subscribes to signals:
    - result_ready(t, rbfi, bfi) each successful frame
    - frame_ready(np.ndarray) every N-th frame for display
    - state_changed(str) state machine transition
    - error(str) fatal error
    """
    result_ready = pyqtSignal(float, float, float)

```

```

        frame_ready = pyqtSignal(np.ndarray)
        state_changed = pyqtSignal(str)
        error = pyqtSignal(str)
        stats_updated = pyqtSignal(dict) # {processed, dropped, fps}

    def __init__(self, source: FrameSource, session: SCOSSession):
        super().__init__()
        self.source = source
        self.session = session
        self.frame_queue = queue.Queue(maxsize=20)
        self._stop_event = threading.Event()
        self._cam_thread = None
        self._proc_thread = None
        self._t0 = None
        self._dropped = 0

        def start(self):
            self.source.open()
            self._t0 = time.monotonic()
            self._stop_event.clear()
            self._cam_thread = threading.Thread(target=self._capture_loop, daemon=True)
            self._proc_thread = threading.Thread(target=self._process_loop, daemon=True)
            self._cam_thread.start()
            self._proc_thread.start()
            self.session.start()

        def stop(self):
            self._stop_event.set()
            if self._cam_thread: self._cam_thread.join(timeout=2)
            if self._proc_thread: self._proc_thread.join(timeout=2)
            self.source.close()

        def _capture_loop(self):
            while not self._stop_event.is_set():
                try:
                    frame = self.source.get_frame(timeout_s=2.0)
                except Exception as e:
                    self.error.emit(str(e))
                return
            try:
                self.frame_queue.put(frame, block=False)
            except queue.Full:
                # queue is full - discard the OLDEST frame
                try:
                    self.frame_queue.get_nowait()
                except queue.Empty:
                    pass
            self.frame_queue.put_nowait(frame)
            self._dropped += 1
            except queue.Empty:
                pass

        def _process_loop(self):
            while not self._stop_event.is_set():
                try:
                    frame = self.frame_queue.get(timeout=0.5)
                except queue.Empty:
                    continue

                t = time.monotonic() - self._t0
                try:
                    self.session.feed_frame(frame, t)
                except Exception as e:
                    self.error.emit(f"Processing error: {e}")
                continue

    # If session is in MEASURING state - send result

```

```

s = self.session.state
if s.times and s.rbfi:
self.result_ready.emit(s.times[-1], s.rbfi[-1], s.bfi[-1])

```

Layer 5: GUI

Here we connect PyQt6. GUI listens to pipeline signals and updates widgets.
Main rule: update the graph NOT on every signal, but via QTimer every 100 ms.

```

# gui/main_window.py
from PyQt6.QtCore import QTimer

class MainWindow(QMainWindow):
    def __init__(self):
        ...
        # Result accumulator between graph updates
        self._pending_results = []
        # Graph update timer
        self._plot_timer = QTimer(self)
        self._plot_timer.setInterval(1000) # 1000 ms
        self._plot_timer.timeout.connect(self._flush_to_plot)
        self._plot_timer.start()

    def _on_result_ready(self, t, rbfi, bfi):
        # Do not draw immediately - accumulate in buffer
        self._pending_results.append((t, rbfi))

    def _flush_to_plot(self):
        if not self._pending_results:
            return
        ts, rs = zip(*self._pending_results)
        self.plot_widget.append_many(list(ts), list(rs))
        self._pending_results.clear()

```

10. Phase 4 — Extension and Polishing

Duration: continuously after Phase 3. Goal: make the project reliable and convenient for colleagues.

Step 4.1 — CI on GitHub Actions

Run ruff + mypy + tests automatically on every push to the repository.

```

# .github/workflows/ci.yml
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ${{ matrix.os }}
    strategy:
      matrix:
        os: [ubuntu-latest, windows-latest]
        python: ["3.11", "3.12"]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python }}
      - run: pip install -e ".[dev]"
      - run: ruff check .
      - run: mypy core/
      - run: pytest --cov=core
    env: { QT_QPA_PLATFORM: offscreen }

```

Step 4.2 — Extended Tests

- pytest-qt — for GUI signal tests

- hypothesis — property-based tests for `scos_math` ($\kappa^2 \geq 0$, gain monotonicity, etc.)
- `pytest-benchmark` — for tracking performance (if processor gets slower, you will see it)

Step 4.3 — README

File `README.md` in the project root. What should be there:

- What SCOS measures (1 paragraph for an outsider)
- Lab: Optical Neuroimaging Lab, Bar-Ilan University
- Equipment checklist: camera (model), laser, Arduino
- Installation instructions (one command if possible)
- Run instructions
- Where to find sample data
- Where to direct questions

Step 4.4 — Logging

Replace all `print()` with logging. Log writes to file and console.

```
# core/logging_setup.py
import logging
from pathlib import Path

def setup_logging(session_folder: Path):
    log_file = session_folder / "scos.log"
    logging.basicConfig(
        level=logging.INFO,
        format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
        handlers=[
            logging.FileHandler(log_file),
            logging.StreamHandler(),
        ]
    )
    return logging.getLogger("scos")
```

Step 4.5 — Claude Code Hooks

File `.claude/settings.json`:

- `PostToolUse`: when `core/scos_math.py` is edited, run tests against `offline_reference`
- `PreToolUse`: block git commit if math was changed but the reference was not updated
- `PostToolUse`: warn if `core/camera_source.py` gets an import from `gui`

11. Glossary

Simple explanations of all the terms that appear in this plan.

Thread — An independent line of execution inside one program. A program can have many threads running in parallel. In our case: one thread captures frames from the camera, another processes them, a third draws the GUI.

Queue — A container for passing data between threads. One thread puts (put), another takes (get). `queue.Queue` in Python is thread-safe — no race conditions.

GUI freeze — When the main thread of a program does something time-consuming and cannot respond to the user. The window stops reacting; Windows shows 'Program not responding'.

Event loop — An infinite loop in the GUI framework (PyQt). It waits for events (clicks, mouse movements) and calls handlers. If a handler runs for a long time, the event loop freezes.

pyqtSignal — Qt's mechanism for communicating between threads. One thread emits a signal (with data), the main thread receives it in a slot. Thread-safe.

QTimer — A Qt timer that emits a timeout signal every N ms. Used for regular GUI updates (graph, status).

State machine — A program with an explicit set of states (IDLE, DARK_CAL, MEASURING) and transitions between them. Helps structure complex logic.

dataclass — A Python way to create a simple class for storing data. The `@dataclass` decorator eliminates the need to write `__init__` and other methods.

Abstraction (ABC) — A class with methods that have no implementation, defining an interface. Concrete subclasses implement the methods. In our case `FrameSource` is the abstraction, `CameraSource` and `FolderSource` are implementations.

uniform_filter (scipy) — Local averaging filter: for each pixel computes the mean over a window x window area. Vectorized (fast), equivalent to MATLAB `imboxfilt`.

Lint — A program that finds errors and style problems in code without running it. `Ruff` is a modern fast linter for Python.

Type checker — A program that verifies you are using correct types (int where int, str where str). `mypy` for Python.

pre-commit hook — A program that runs before every git commit. If it finds a problem — the commit is blocked.

CI (continuous integration) — A server that runs your tests automatically on every push to the repository. GitHub Actions is the most popular.

Lock-file — A file with exact versions of all libraries. `requirements.lock` = 'pinned versions'. Guarantees your environment tomorrow will be the same as today.

pyproject.toml — Modern file with Python project metadata: name, version, dependencies, `ruff/mypy/pytest` configs. Replaces `requirements.txt` + `setup.py` + `setup.cfg`.

Downsampling — Reducing the number of points in data. For a 200k-point graph: show only every 40th point → 5000 points, draws quickly.

Producer-consumer pattern — Architecture pattern: one thread (producer) puts work into a queue, another (consumer) takes and executes it. Very popular for real-time.

GenTL / GenICam — Genicam standards for machine vision. `pypylon` implements them for Basler cameras.

TIFF / HDF5 — Formats for scientific data. TIFF — for individual images, HDF5 — for large tables with append support (adding data without rewriting).

12. Checklist: What to Do Today and Tomorrow

A concentrate of the entire plan as tasks that can be checked off.

Today (Phase 0 — Reference)

- ☐ Find one good recording in the lab archive (1-5 minutes)
- ☐ Copy it to a separate folder `scos-python/sample_data/`
- ☐ Record parameters: `gain_db`, `exposure_us`, `frame_rate`, `pixel_format`, `sat_capacity`
- ☐ Run the existing offline code on this recording
- ☐ Save output as `tests/data/offline_reference.npz`
- ☐ Save the first 20 frames as `tests/data/sample_frames.npz`
- ☐ Write in README which recording was used and why

Tomorrow (Phase 1 — Architecture on Paper)

- ☐ Create `docs/realtime_architecture.md`
- ☐ Answer in writing all questions from section 7 of this plan

- ☐ Draw a diagram of threads and queues (can be done by hand, then photographed)
- ☐ Draft the core/ + tests/ folder structure
- ☐ Define data structures ProcessingResult, SessionConfig, SessionState
- ☐ Run bench_processor.py on a 700×700 frame, record the result
- ☐ Decide: do we fit in 50 ms? If not — what is the strategy?
- ☐ Discuss with supervisor: normalization for long recordings, disk space limit

This Week (Phase 2 — Tools)

- ☐ Create pyproject.toml
- ☐ Run pip-compile, get requirements.lock, commit it
- ☐ Install ruff, mypy, pre-commit
- ☐ Create .pre-commit-config.yaml, run pre-commit install
- ☐ Update README with installation instructions

Next 2-4 Weeks (Phase 3 — Code)

- ☐ Layer 1: write core/scos_math.py + test against offline_reference
- ☐ Layer 2: write core/frame_source.py + folder_source.py + tests
- ☐ Layer 2: write core/camera_source.py (adaptation from camera.py)
- ☐ Layer 3: write core/session.py with state machine + tests
- ☐ Layer 4: write core/pipeline.py with threads and queues
- ☐ Layer 4: integration test of pipeline on FolderFrameSource
- ☐ Layer 5: rework gui/main_window.py for new pipeline
- ☐ Layer 5: add progress bar for calibration, pop-up dialogs
- ☐ Layer 5: add downsampling to plot_widget for long recordings
- ☐ Implement progressive saving to HDF5
- ☐ Implement loading G[DU/e] from table by camera SN

When the Main Part Works (Phase 4 — Polishing)

- ☐ Set up GitHub Actions CI
- ☐ Add pytest-qt and pytest-benchmark
- ☐ Write a full README
- ☐ Replace print() with logging everywhere
- ☐ Configure .claude/settings.json hooks
- ☐ Run 1 hour of real recording and compare with offline
- ☐ Document all answered questions (ADR documents)

13. Clarifications After Review

Two important architectural decisions:
 13.1 — about the Pylon capture strategy.
 13.2 — about what to do when processing cannot keep up with the camera.

13.1 Pylon Capture Strategy

Why LatestImageOnly is Dangerous for SCOS

With LatestImageOnly, Pylon keeps only the most recent frame. If the capture loop falls slightly behind, Pylon silently discards old frames — no error, no warning, no log. The Python code continues running as if nothing happened.

For SCOS this is critical, because the $1/\kappa^2$ time series is fed into FFT to extract the pulse frequency. FFT assumes the signal is uniformly sampled (one point every 50 ms at 20 Hz, no gaps). If frames are silently lost — the sampling is no longer uniform, and FFT will show the wrong frequency. A real pulse of 72 bpm may show up as 48 bpm, with no obvious failure. This is the worst kind of bug: silent, invisible, and corrupts the scientific result.

What to use instead:

For SCOS, switch to `GrabStrategy_OneByOne` and configure two things:

```
self.camera.MaxNumBuffer.Value = 20 # how many frames Pylon can hold
```

```
self.camera.StartGrabbing(pylon.GrabStrategy_OneByOne)
```

Switch to `GrabStrategy_OneByOne` (FIFO queue, all frames are kept up to buffer size) and explicitly set the buffer size: `self.camera.MaxNumBuffer.Value = 20`. One 700×700 Mono12 frame is about 1 MB; a buffer of 20 frames is about 20 MB.

Detecting losses when they do happen

Even with `OneByOne`, sustained slowdown will eventually fill the buffer and frames will start being lost anyway. The key difference from `LatestImageOnly` — we can now detect the loss and react via `result.GetNumberOfSkippedImages()`. This counter returns how many frames were discarded between the previous successful grab and the current one. This is the input signal for the user decision in section 13.2.

Three buffering layers that are often confused

- Layer 1 — Pylon driver buffer (SDK memory between camera and Python; controlled by `MaxNumBuffer` and `GrabStrategy`)
- Layer 2 — Python `frame_queue` (our code's memory between the capture thread and processing thread; controlled by `queue.Queue(maxsize=...)`)
- Layer 3 — current processor work (math currently running on one frame; controlled only by CPU time)

Increasing the Pylon buffer does not fix a slow processor — it just gives a bit more time before losses start. Increasing the Python queue also does not fix it. The only real solution for sustained load is to speed up the math, reduce the ROI, lower FPS, or ensure processing runs in its own separate thread.

13.2 Overload Behavior — What to Do When Processing Cannot Keep Up

Principle

Frames MUST NOT be silently discarded. If the system cannot keep up — the user must be informed and given a choice of how to resolve the situation. This rule is the same for all states — preview, calibration, measurement. The main reason — FFT (see section 13.1).

When to detect — high-water mark, not already-failed

Do not wait until frames actually start being lost. Monitor `frame_queue`. When fill reaches ~80% of size — raise a warning. This is called a high-water mark. This gives the user time to react BEFORE data is corrupted, not after. With queue size 20 this gives about 800 ms to react.

What to do when the high-water mark triggers

- Pause capture — do not stop or reset. Pause preserves all calibration state and allows immediate resumption.
- Show a non-blocking dialog with four options (see below).
- On timeout (5-10 seconds of silence) apply a safe default action — switch to 'save only' mode.

Four options for the user (in warning window), in order of increasing disruption:

- Reduce ROI — fewer pixels per frame to process, immediately speeds things up. Best first choice.
- Reduce FPS — lower camera frame rate; changes time resolution of blood flow signal, affects FFT accuracy and BPM.
- Save only, do not process — skip κ^2 computation; write raw frames to disk; process offline later. Safe fallback option.
- Process only, do not save — skip disk writing; continue computing κ^2 . Risky.

GUI feedback — do not surprise the user

Add two small indicators to the GUI that permanently show the pipeline state:

- fill bar for frame_queue (from 0% to 100%)
- dropped frame counter (increments by GetNumberOfSkippedImages from 13.1)

The goal — teach the user what 'system under stress' looks like, instead of throwing a dialog at them as a surprise. And also give the scientific supervisor objective numbers for the paper: 'during the recording, 0 frames were dropped' — this is important information about data quality.

For the future

If 'always fresh' mode is ever needed in preview (for example, to quickly show the user a focus/contrast indicator where latency matters more than completeness), it can be added as a separate option.

Implementation — just add a second path in _capture_loop with a state check. For now it is not needed: preview is short, κ^2 processing does not run in it, and the queue is normally almost empty.

Conclusion

Key principles that will help with any surprises:

- Do not write GUI first. GUI is the last layer, not the first.
- Do not drop frames.
- Heavy work — not in the main thread. Never.
- Threads communicate through queues, not through shared variables.
- Tests for each layer are written BEFORE the next layer.
- Small commits are better than large ones.
- If in doubt about architecture — ask, but first write it down on paper.